CLASE 1 · PROGRAMACIÓN EN BASH

```
usuario@clase:~$ ./clase1.sh
```

Introducción a Bash

Shells, primeros scripts y el entorno de Bash

Basado en los capítulos 1 a 3 de *Bash Guide for Beginners*, de Machtelt Garrels

CAPÍTULO 1

Bash y los scripts de Bash

Qué es un shell, qué ofrece Bash y cómo interpreta y ejecuta lo que escribimos.

01

1.1 · PROGRAMAS DE SHELL

¿Qué es un shell?

Usuario o script
escribe o guarda comandos

`comandos` →

Shell
los interpreta línea por línea

`llamadas` →

Kernel
ejecuta el trabajo real

Interpretada, no compilada

El shell lee el script línea por línea y busca cada comando en el sistema. Un compilador, en cambio, convierte un programa en un ejecutable.

También es tu entorno

Además de pasar comandos al kernel, el shell ofrece un entorno de usuario configurable con archivos de recursos.

Una familia de shells

Shell	Nombre	En pocas palabras
sh	Bourne Shell	El original. Pocas funciones; presente en todo Linux por compatibilidad.
bash	Bourne Again Shell	El shell GNU, estándar en Linux. Superconjunto de sh: lo que corre en sh corre en bash.
csh	C Shell	Sintaxis parecida al lenguaje C; a veces lo piden los programadores.
tcsh	TENEX C Shell	Una versión de csh mejorada en facilidad de uso y velocidad.
ksh	Korn Shell	Superconjunto de sh, apreciado por quienes vienen de UNIX.

Ver las shells disponibles: `cat /etc/shells` · cambiar de shell: escribir su nombre, p. ej. `tcsh`

1.2 · VENTAJAS DE BASH

¿Por qué Bash?

Compatible con sh, con ideas tomadas de ksh y csh, y software libre del proyecto GNU.

Puede ejecutar la mayoría de los scripts de sh sin modificarlos.

- Edición de la línea de comandos
- Historial de tamaño ilimitado
- Control de trabajos (jobs)
- Funciones y alias
- Arrays indexados
- Aritmética entera, bases 2 a 64

1.2.2 · ARCHIVOS DE INICIO

¿Qué archivos lee Bash al iniciar?

Shell de login interactiva

Después de autenticarse con usuario y contraseña.

```
/etc/profile
~/.bash_profile | ~/.bash_login | ~/.profile
~/.bash_logout  (al salir)
```

Interactiva, sin login

Por ejemplo, al abrir una terminal desde el menú.

```
~/.bashrc
```

No interactiva

Así corren todos los scripts.

```
archivo indicado en $BASH_ENV
```

Invocada como sh

Imita al Bourne shell histórico.

```
/etc/profile
~/.profile
```

1.3 · EJECUCIÓN DE COMANDOS

Cómo ejecuta Bash un programa

bash
proceso padre

`fork` →

copia de bash
hijo: mismo entorno, otro PID

`exec` →

programa nuevo
reemplaza al hijo, p. ej. ls

fork-and-exec

El hijo hereda el entorno del padre: variables, entrada y salida, prioridad. Así se crean todos los procesos en UNIX y Linux.

Comandos internos (built-ins)

cd, **echo**, **export**, **set**... viven dentro de la shell: se ejecutan sin crear un proceso nuevo.

Scripts

Bash crea un subshell que lee el script línea por línea; el padre espera a que termine.

1.4.1.7 · BÚSQUEDA DE COMANDOS

¿Dónde busca Bash un comando?

1

¿Es una función?

Primero revisa las funciones definidas.

→

2

¿Es un built-in?

Luego, los comandos internos de la shell.

→

3

¿Está en el PATH?

Recorre esos directorios; una tabla hash recuerda las rutas.

→

4

No encontrado

Muestra un error y devuelve el código 127.

Si el nombre contiene una barra, como **./script.sh**, Bash no busca: ejecuta esa ruta directamente.

Del texto al comando, paso a paso

1. Lee la entrada: de un archivo, una cadena o la terminal.
2. La divide en palabras y operadores, respetando las comillas; expande los alias.
3. Analiza los tokens y arma comandos simples y compuestos.
4. Realiza las expansiones de la shell.
5. Aplica las redirecciones.
6. Ejecuta el comando.
7. Opcionalmente espera a que termine y recoge su código de salida.

Comando simple

```
touch file1 file2 file3
```

Comando compuesto (tubería)

```
ls | more
```

¿Qué hace bueno a un script?

1. Corre sin errores.
2. Cumple la tarea para la que fue pensado.
3. Su lógica es clara y evidente.
4. No hace trabajo innecesario.
5. Es reutilizable.

Antes de escribir, preguntarse:

- ¿Necesito información del usuario o de su entorno? ¿Cómo la guardo?
- ¿Hay que crear archivos? ¿Dónde, con qué permisos y dueños?
- ¿Qué comandos usaré, y existen en todos los sistemas destino?
- ¿Hay que avisar algo al usuario? ¿Cuándo y por qué?

1.5.5 · EJEMPLO

Un primer script: mysystem.sh

```
#!/bin/bash
clear
echo "Información de mysystem.sh."
echo "Hola, $USER"
echo "Hoy es `date`, semana `date +"%V"`."
echo "Usuarios conectados:"
w | cut -d " " -f 1 - | grep -v USER | sort -u
echo "Esto es `uname -s` en un `uname -m`."
echo "Tiempo en funcionamiento:"
uptime
echo "¡Eso es todo!"
```

`#!/bin/bash`

Primera línea: indica qué shell ejecuta el script.

`$USER`

Se reemplaza por el nombre del usuario.

`` `date` ``

Inserta la salida del comando date.

`echo / printf`

echo imprime texto; printf permite dar formato.

CAPÍTULO 2

Escribir y depurar scripts

Crear un script, elegir su intérprete, comentarlo, ejecutarlo y encontrar sus errores.

02

2.1 · CREAR Y EJECUTAR

Escribir, dar permisos, ejecutar

1

Escribir

Cualquier editor de texto sirve. Nombre descriptivo, terminado en .sh, que no choque con otro comando.

```
which -a script1.sh
```

2

Dar permisos

Para correr como un comando, el script necesita permiso de ejecución.

```
chmod u+x script1.sh
ls -l script1.sh
```

3

Ejecutar

Por su nombre si su carpeta está en el PATH; si no, con la ruta.

```
./script1.sh
```

Consejo: guardar los scripts en ~/scripts y agregar esa carpeta al PATH: `export PATH="$PATH:~/scripts"`

2.1.3 · EJECUTAR EL SCRIPT

¿Subshell o shell actual?

`./script1.sh`

Corre en un **subshell**. Las variables, funciones y alias que cree se pierden al terminar.

```
$ ./script1.sh
$ echo $VALUE
#  (vacío)
```

`source script1.sh`

Corre en el shell **actual** y no necesita permiso de ejecución. Los cambios quedan en el entorno.

```
$ source script1.sh
$ echo $VALUE
9
```

source y **.** (punto) son sinónimos. Para elegir el intérprete explícitamente: **bash script.sh**, **sh script.sh** o **bash -x script.sh**.

2.2 · BASES DEL SCRIPT

Shebang y comentarios

```
#!/bin/bash
# Limpia la terminal, saluda y muestra
# los usuarios conectados.
clear                  # limpia la ventana
echo "Hola, $USER!"    # $ lee la variable
w                      # quién está conectado
COLOUR="black"         # variable local
VALUE="9"              # variable local
echo "Texto: $COLOUR"
echo "Número: $VALUE"
```

#!

Los dos primeros caracteres de la primera línea, seguidos de la ruta del intérprete. Sin líneas en blanco antes. En este curso: **#!/bin/bash**

#

Todo lo que sigue a # en una línea se ignora. Los comentarios ayudan a otros lectores… y a uno mismo, meses después.

2.3 · DEPURACIÓN

Depurar un script

Todo el script

```
$ bash -x script1.sh
+ clear
+ echo 'Hola, willy!'
Hola, willy!
+ COLOUR=black
+ VALUE=9
```

Solo una parte

```
set -x   # activa la depuración
w
set +x   # la desactiva
```

O en la primera línea: **#!/bin/bash -xv**

Corta	Larga	Efecto
set -x	set -o xtrace	Muestra cada comando antes de ejecutarlo
set -v	set -o verbose	Muestra las líneas tal como se leen
set -f	set -o noglob	Desactiva la expansión de nombres de archivo

CAPÍTULO 3

El entorno de Bash

Archivos de inicio, variables, comillas, expansiones, alias y opciones: todo lo que da forma a la shell.

03

3.1 · ARCHIVOS DE INICIALIZACIÓN

Dónde se configura la shell

Para todo el sistema

`/etc/profile`	Variables como PATH, USER, MAIL, HOSTNAME e HISTSIZE.
`/etc/bashrc`	Funciones y alias propios de Bash.

Tras editar un archivo: volver a iniciar sesión o recargarlo con **source ~/.bashrc**.

Para cada usuario

`~/.bash_profile`	El preferido para el entorno personal.
`~/.bash_login`	Se lee si no existe .bash_profile.
`~/.profile`	Si no existen los dos anteriores.
`~/.bashrc`	Shells sin login, como las terminales gráficas.
`~/.bash_logout`	Instrucciones al cerrar sesión.

3.2.2 · CREAR VARIABLES

Crear variables: NOMBRE="valor"

Así sí

```
$ MYVAR1="2"
$ echo $MYVAR1
2
$ full_name="Franky M. Singh"
$ echo $full_name
Franky M. Singh
$ unset MYVAR1 full_name
```

Así no

```
$ MYVAR1 ="2"
bash: MYVAR1: command not found
$ MYVAR-2="2"
bash: MYVAR-2=2: command not found
$ export 1number=1
bash: export: `1number=1': not a
valid identifier
```

Sin espacios alrededor del =

Distingue mayúsculas

No empieza con un dígito

Valor entre comillas

3.2.3 · EXPORTAR VARIABLES

Variables locales y de entorno

```
$ full_name="Franky M. Singh"
$ bash
$ echo $full_name
#  vacío: no fue exportada
$ exit
$ export full_name
$ bash
$ echo $full_name
Franky M. Singh
$ export full_name="Charles the Great"
$ exit
$ echo $full_name
Franky M. Singh
```

Local

Solo existe en el shell actual. **set** lista todas las variables.

De entorno (global)

La heredan los procesos hijos. Se crea con **export VAR="valor"**; **env** o **printenv** las listan.

Solo hacia abajo

Un hijo puede cambiar lo que heredó, pero el cambio no vuelve al padre.

3.2.4 · VARIABLES RESERVADAS

Variables que Bash ya conoce

`HOME`	Directorio personal; destino por defecto de cd y de ~	`PATH`	Directorios, separados por :, donde se buscan comandos.
`PS1`	El prompt principal.	`PS2`	El prompt secundario; por defecto "> ".
`PWD`	Directorio de trabajo actual.	`OLDPWD`	Directorio anterior, antes del último cd.
`IFS`	Caracteres que separan palabras.	`RANDOM`	Un entero al azar entre 0 y 32767 en cada lectura.
`SHLVL`	Sube en uno con cada nueva instancia de Bash.	`UID`	ID numérico del usuario real.

Parámetros especiales

`$1 $2 …`	Parámetros posicionales: los argumentos
`$#`	Cantidad de parámetros posicionales
`"$@"`	Todos, cada uno como palabra aparte
`"$*"`	Todos, en una sola palabra
`$?`	Código de salida del último comando
`$$`	PID de el shell actual
`$!`	PID del último proceso en segundo plano
`$0`	Nombre de el shell o del script

```
#!/bin/bash
# pos.sh
echo "Primero: $1"
echo "Segundo: $2"
echo "Total: $#"
```

```
$ ./pos.sh uno dos tres
Primero: uno
Segundo: dos
Total: 3
$ ls noexiste; echo $?
1
```

3.3 · COMILLAS

Comillas: quitar el significado especial

Con la variable **date=20021226**:

\

Barra invertida

Protege solo el carácter siguiente.

```
$ echo \$date
$date
```

' '

Comillas simples

Todo es literal. No puede haber una comilla simple adentro.

```
$ echo '$date'
$date
```

" "

Comillas dobles

Todo literal excepto $, las comillas invertidas y \.

```
$ echo "$date"
20021226
```

Además: **$'...'** interpreta escapes al estilo ANSI-C como \n o \t, y **$"..."** traduce la cadena según la configuración regional.

Expansiones: lo que Bash reescribe

#	Expansión	Ejemplo	Resultado
1	Llaves	`echo sp{el,il,al}l`	spell spill spall
2	Tilde	`cd ~/docs`	~ se vuelve el valor de $HOME
3	Parámetros y variables	`echo ${SHELL}`	/bin/bash
4	Sustitución de comandos	`echo "Hoy: $(date)"`	la salida de date
5	Aritmética	`echo $(( 365 * 24 ))`	8760
6	División en palabras	`según $IFS`	separa resultados sin comillas
7	Nombres de archivo	`ls *.sh`	los archivos que coinciden

En este orden. También existe la sustitución de procesos, **<(lista)**, y al final se eliminan las comillas.

3.5 · ALIAS

Alias: atajos para comandos

```
$ alias dh='df -h'
$ dh
Filesystem  Size  Used Avail Use%
/dev/hda7   1.3G  272M 1018M  22%
/dev/hda2    13G  8.7G  3.7G  70%
$ unalias dh
$ dh
bash: dh: command not found
```

- ✓ **alias** sin argumentos lista los que están definidos.
- ✓ Solo valen en el shell actual; para que sean permanentes, van en **~/.bashrc**.
- ✓ No se heredan a los procesos hijos y, por defecto, no se expanden en scripts.
- ✓ Conviene definirlos en una línea propia.
- ✓ Para casi todo, las **funciones** son preferibles.

Opciones de el shell con set

```
$ set -o noclobber
$ touch test
$ date > test
bash: test: cannot overwrite existing file
$ set +o noclobber

$ set -u
$ echo $VAR
bash: VAR: unbound variable
```

set -o muestra todas las opciones y su estado. **-** activa, **+** desactiva.

noclobber

No sobrescribir archivos con >

nounset (-u)

Error al usar una variable no definida

noglob (-f)

No expandir *, ? ni [ ]

xtrace (-x)

Traza de ejecución, como ya vimos